

Agent Execution Control Planes: When “Sandbox”, “Gateway”, and “Policy” Ship as Unauthenticated Code Execution

Anonymous

Anonymous

Abstract. The “execution control plane” is an emerging architectural class in the AI-agent ecosystem: services named *agent-exec-gateway*, *agent-sandbox*, *code-sandbox*, *policy engine*, or *tool-gateway* that are *supposed* to constrain what an AI agent may execute—by policy, approval, and sandbox isolation. We present an empirical, runtime-verified security campaign over the freshly-published population of this class. Scanning ~500 newly-published MCP/A2A/agent repositories across five attack-surface dimensions, we confirm and responsibly disclose **4 exploitable vulnerabilities** whose root cause is a single recurring pattern we term *control-plane trust inversion*: the very component whose name promises restriction is shipped as an *unauthenticated, direct code-execution endpoint*. In two cases an attacker with network access injects an allow-all policy and triggers arbitrary command/Python execution with output read-back, bypassing the service’s own approval gate; one ships a hardcoded credential plus two unauthenticated services; and one exposes unauthenticated policy-rewrite and self-approval on a human-approval control plane. We contrast these with a control group of seven mature implementations (fail-closed boot, HMAC constant-time compare, single-use approval, VM/gvisor isolation) to show the ecosystem is *two-sided*: established/enterprise agent infrastructure is hardened, while freshly-published 0-star “control-plane” repositories repeatedly invert the trust model. We argue this is an agent-ecosystem-specific gap—distinct from the MCP-tool hardening documented in prior work—because the naming (*sandbox/gateway/policy*) creates a false assurance that a restriction layer exists. We provide detection signatures, honest deployment-boundary analysis, and hardening guidance.

Keywords: Agent Execution Control Plane · Unauthenticated Code Execution · CWE-306 · Agent Sandbox · Policy Bypass · Ecosystem Security

1 Introduction

AI agents increasingly carry *code-execution authority*: they run commands, evaluate Python, and manipulate hosts through a stack of tools and servers. To keep these agents safe, a new component class has appeared across the ecosystem—the *execution control plane*. Its members adopt names that promise restriction—*agent-exec-gateway*, *agent-sandbox*, *code-sandbox*, *policy engine*, *tool-gateway*—and are described as constraining agent actions by policy matching, human approval, or sandbox isolation.

Prior systematic work has documented tens of exploitable vulnerabilities in the *MCP tool* ecosystem: command injection (CWE-78), SSRF (CWE-918), read-only gate bypass (CWE-281), and unauthenticated exposure (CWE-306), concentrated in low-star repositories [2,1,5]. Those corpora were built from *established* servers and focused on tool primitives. A distinct and under-measured surface is the *control-plane layer* itself—the services whose entire purpose is to gate execution. When a control plane is unauthenticated, an attacker does not merely reach a powerful tool; the attacker can *rewrite the policy*, *self-approve*, and *execute arbitrary commands*, turning the supposed guardrail into the primary RCE primitive.

This paper presents a focused empirical study over the freshly-published population of this class (2026-09-24 to 2026-10-02). Our contributions:

1. **A new defect class**—*control-plane trust inversion*: the component named for restriction is shipped as an unauthenticated, direct code-execution endpoint (CWE-306), enabling policy bypass and arbitrary command/Python execution.
2. **Three runtime-confirmed disclosures** across three 0-star repositories, with honest deployment-boundary analysis (default loopback vs. 0.0.0.0 Docker).
3. **A two-sided maturity finding**: a control group of seven hardened implementations (fail-closed boot, constant-time compare, single-use approval, VM isolation) shows the ecosystem is *not uniformly* insecure—established infrastructure hardened, while 0-star “control-plane” repos invert the trust model.
4. **Detection signatures and mitigation patterns** for triaging this class.

2 Background and Related Work

2.1 The agent execution stack

A typical agent execution stack has several layers: the *model* (Claude/Codex/GPT), a *client/framework* that turns tool calls into actions, *tools* that perform concrete operations, and—increasingly—a *control plane* that decides whether an action is allowed. Control planes differ from tools: they exist to *enforce a boundary* (policy, approval, sandbox), not to perform the operation itself. This is why their security posture matters doubly: a broken control plane both fails to constrain and exposes an execution primitive.

2.2 Related empirical audits

Recent systematic audits of the MCP/A2A ecosystems [5,3,4] catalogued command injection, SSRF, read-only-gate bypass, credential-theft oracles, and principal-spoofing in *established* and *fresh* tool servers. Our work differs in two ways: (a) we target the *control-plane layer* specifically rather than tool primitives, and (b) we show the recurring pattern is *trust inversion*—naming promises restriction while the implementation exposes direct execution—which is an agent-ecosystem-specific gap. The MCP-tool hardening documented in prior work does not automatically extend to agent control planes.

3 Threat Model

We assume an attacker who can reach the control plane’s HTTP endpoint. Two deployment postures matter:

- **Loopback (default in most repos):** attacker = local user, co-located MCP client, or same-host process. Impact = local/reachable arbitrary command execution.
- **0.0.0.0 (shipped in Dockerfiles):** attacker = any remote peer. Impact = remote unauthenticated RCE.

The attacker’s objective against a control plane is *authority escalation*: (1) read the configuration (hosts, policies, approvals, execution outputs); (2) rewrite the policy to allow all; (3) self-approve pending approvals; (4) register new execution hosts (including SSH targets with identity files); (5) trigger arbitrary command execution and read output. We consider the control plane *itself* as the attack surface, not the tools it gates.

4 Attack Surface and the Inversion Pattern

4.1 Control-plane trust inversion

The recurring root cause across all confirmed findings is *trust inversion*:

```
control plane (named sandbox/gateway/policy)
      ↓ should be “the constraint that limits an agent”
      ↓ actually ships as “unauthenticated + arbitrary code exec + attacker-rewritable”
```

Three concrete sub-patterns:

1. **Control plane rewritable unauthenticated.** (agent-exec-gateway) The `/api/policies` endpoint accepts an allow-all rule without auth; `/api/hosts` registers execution hosts; `/api/approvals/.../approve` self-approves. An attacker injects `{client:'*',host:'*',command:'*',decision:'allow'}` then triggers any command.
2. **“Sandbox” = bare subprocess.** (agent-sandbox-runtime) Despite the name, `POST /v1/run` does `subprocess.run(["python", "-c", value])` with no isolation, and the Dockerfile binds 0.0.0.0.
3. **Security service itself unguarded.** (e2b-mcp-server) A companion “guard” HTTP API binds 0.0.0.0 with `Access-Control-Allow-Origin: *` and no auth, exposing blocklist manipulation and operational data.

4.2 Audit checklist for this class

When triaging an execution control plane, check in order:

1. **Bind default:** loopback (127.0.0.1) vs. 0.0.0.0 (esp. in Dockerfile/README deploy command).
2. **Every `/api/*` route auth:** is there a central middleware/Depends, or bare handlers?
3. **Execution primitive:** `subprocess/python -c/spawn` reachable from an unauthenticated endpoint? argv fully attacker-controlled?
4. **Control-plane write:** can an unauthenticated caller add policies/hosts/approvals?
5. **Self-approval:** is there an approve endpoint the execution submitter can also call?

5 Findings

We confirmed and responsibly disclosed three vulnerabilities. Table 1 summarizes; each is runtime-verified on an isolated loopback instance with marker commands only, and disclosed via public GitHub issue.

Table 1. Confirmed vulnerabilities in agent execution control planes (2026-10-02 campaign).

Repo	Defect
truong51972/agent-exec-gateway	Unauth policy/host/approval endpoints + arbitrary <code>create_subprocess_exec(*arg</code>
nagasail7bce-rgb/agent-sandbox-runtime	Unauth <code>POST /v1/run</code> → <code>subprocess.run(["python", "-c", value])</code> , no is
JestonBatson/governed-policy-engine	Unauth <code>PUT /v1/policies</code> (rewrite) + <code>POST /v1/approvals</code> (self-grant)
ishakking642-eng/e2b-mcp-server	Hardcoded API key (CWE-798) + two unauth 0.0.0.0 services (CWE-306)

5.1 agent-exec-gateway (CWE-306)

A FastAPI control plane where *every* `/api/*` route is unauthenticated, and the control-plane writes are themselves the enablers:

- `POST /api/policies` {`client:'*'`, `host:'*'`, `command:'*'`, `decision:'allow'`} → 201, injects an allow-all rule that makes every subsequent execution `decision=allow` (bypassing the default ASK human approval).
- `POST /api/hosts` registers a local/SSH execution host (SSH hosts store `identity_file` path).
- `POST /api/executions` {`host:'local'`, `argv:[...]`} → 200 `status=completed` `decision=allow`, `stdout` read back.
- `POST /api/approvals/{id}/approve` lets the submitter self-approve a pending execution.

Runtime evidence: `unauth POST /api/executions` with `argv=["python", "-c", "print('__AEG_RCE_VERIFY_44e6f8__')"]` returned `stdout: __AEG_RCE_VERIFY_44e6f8__`, `decision=allow`, no approval. **Honest scope:** default `bind_host=127.0.0.1` ⇒ local/reachable RCE; deployed to 0.0.0.0 or behind a proxy, remote RCE.

5.2 agent-sandbox-runtime (CWE-306)

Named *sandbox*, but `POST /v1/run` executes arbitrary Python with no isolation:

```
1 # app/service.py
2 process = subprocess.run(["python", "-c", value],
3                           capture_output=True, text=True, timeout=2)
4 # app/main.py (FastAPI)
5 @app.post("/v1/run")
6 def run(req: Request): return service.run(req.value) # no
7 auth, no sandbox
```

Runtime evidence: `POST /v1/run {value:"import socket; print('__ASR_RCE_VERIFY_91f7c3__')"} returned stdout: __ASR_RCE_VERIFY_91f7c3__ <hostname>.` **Honest scope:** the README self-declares this is a reference boundary with authentication listed as a roadmap item (doc-vs-executable); however the Dockerfile binds 0.0.0.0 and the README deploy command is `docker run -p 8000:8000 agent-sandbox-runtime`, which makes the unauthenticated code-exec endpoint *remotely reachable* = RCE on the container host.

5.3 e2b-mcp-server (CWE-798 + CWE-306)

A Daytona sandbox manager MCP server with (a) a real-format Daytona API key hardcoded as the default fallback of `os.getenv('DAYTONA_API_KEY', 'dtn_365790d2...')` (CWE-798); (b) the FastMCP SSE server bound to 0.0.0.0 with no auth (CWE-306), so any peer can call `wake_up_sandbox()` which issues authenticated requests to the operator's workspace; (c) a companion "guard" HTTP server bound to 0.0.0.0 with `Access-Control-Allow-Origin: *` and no auth on any of its 12 routes, enabling unauthenticated blocklist manipulation, feed injection, and operational-data disclosure. **Honest scope:** the Daytona key's validity was not probed; workspace control is conditional on the key being live; the guard impact is blocklist/feed manipulation + disclosure (no command-execution surface), deployment-dependent.

5.4 governed-policy-engine (CWE-306)

A "deterministic authorization and human-approval control plane for AI-enabled systems." Its two security-critical write endpoints are unauthenticated:

- `PUT /v1/policies` any caller can *rewrite all policies* (remove a `require_approval` rule or add an allow-all rule).
- `POST /v1/approvals` any caller can *grant an approval* for an arbitrary (actor, action, resource) (self-approval).
- `POST /v1/decisions` when a request carries the `approval_id`, the engine evaluates to `ALLOW / APPROVAL_GRANTED` (per the repo's own `scripts/approval_demo.py`).
- Dockerfile binds 0.0.0.0; `docker compose up` maps 8000:8000 default bind is public, not loopback.

Runtime evidence: `unauth PUT /v1/policies (require_approval on customer.delete) → 200; unauth POST /v1/approvals {id:"attacker-approved-1", actor_id, action, resource, expires_at} → 200 (approval granted, id returned).` The subsequent `ALLOW` decision leg required `AUDIT_SIGNING_KEY` to be set (a correct fail-closed deployment requirement, enforced by `docker-compose` too), which we did not configure, but the author's own demo script documents that a scoped approval flips `REQUIRE_APPROVAL→ALLOW`. **Honest scope:** impact is *governance bypass* (forcing high-risk AI actions that should require human approval to be treated as allowed),

not code execution; default Docker bind 0.0.0.0 makes the unauth policy/approval write surface remotely reachable.

6 Control Group: the Ecosystem Is Two-Sided

To avoid over-generalizing from three findings, we audited a control group of seven *mature* execution control planes (Table 2). All are hardened:

Table 2. Control group—mature agent control planes (no vulnerabilities).

Implementation	Hardening observed
ib-mcp	fail-closed boot (no TOKEN + no ALLOW_UNAUTH = refuse), HMAC <code>compare_digest</code> , trading opt
mcp-confirm (les-k)	single-use approval + RequestStateBoundary + target re-check at exec time, fail-closed
ai-agent-firewall	AST analyzer + behavioral analysis + runtime guards
agent-ssh-gateway	token store + ownership + audit (Alembic migrations)
agents-sandbox (inoio)	gvisor/VM runtime + network policy + mounts parsing + reap policy
mcp-governance-gateway	<code>token_urls</code> safe 32B + <code>auth.py</code> + audit + confirm
web-agent-gateway	pairing token 32–256 + double-approval + token-class isolation

The contrast is sharp: established/enterprise agent infrastructure consistently ships fail-closed boot, constant-time comparison, single-use approval, and OS-level isolation, while freshly-published 0-star “control-plane” repositories repeatedly invert the trust model. This two-sidedness is the paper’s central empirical finding: the risk is not “agent control planes are insecure” but “*the control-plane name creates false assurance precisely where freshly-published code is least trustworthy.*”

7 Measurement and Method

7.1 Campaign scope

Over 2026-09-24 to 2026-10-02 we scanned five attack-surface dimensions across the freshly-published MCP/A2A/agent ecosystem (~500 repositories triaged): (1) supply-chain poisoning of PyPI/npm mcp packages (444 targets, 0 findings; npm security-hold protects `mcp-server-*` names), (2) agent permission-escaping / `exec-sandbox-gateway` (8 targets, 2 findings), (3) credential storage (3 targets, 0 findings; fail-closed mature), (4) prompt-injection defense (8 targets, 0 findings), (5) protocol/transport implementation (3 targets, 0 findings; official SDK + `mcp-proxy` hardened), plus an expanded execution-control-plane sweep (40 targets, 0 additional findings). Every candidate was triaged via the Section 4.2 checklist; high-signal candidates were runtime-verified on isolated loopback instances with marker commands only.

7.2 Honest limitations

The confirmed-findings hit rate over the full ~ 500 -repo sweep is low ($\approx 0.4\%$); the two execution-control-plane findings came from a *targeted* sweep (8 candidates). This is expected: the pattern is specific (*unauth + control-plane-writable + bare-exec primitive*), not uniformly present. We report the honest numbers, the specific discovery signatures, and the deployment-boundary conditions rather than inflate a broad “ecosystem insecurity” claim. Findings are source+runtime confirmed on loopback; remote reachability depends on the documented default bind (mostly loopback unless the Dockerfile/README overrides to 0.0.0.0).

8 Mitigation

1. **Fail-closed boot:** refuse to start if no token is configured and unauthenticated mode is not explicitly enabled (as `ib-mcp` does).
2. **Central auth on every `/api/*` route:** enforce via middleware/Depends, never per-route omission.
3. **Policy/host writes are admin-only:** an unauthenticated caller must never add policies, hosts, or approvals.
4. **Human-only approval:** no self-approve endpoint reachable by the execution submitter.
5. **Dockerfile binds loopback by default:** require an explicit opt-in (`-host 0.0.0.0`) plus auth before remote exposure.
6. **Real isolation for anything named sandbox:** container/microVM, read-only FS, resource quotas, network policy, dropped privileges (the README guidance the implementation must actually honor).
7. **Don’t let naming create false assurance:** a “sandbox”/“gateway”/“policy” name is a security claim; it must be verified, and 0-star freshly-published instances deserve the same scrutiny as any untrusted network service.

9 Conclusion

Execution control planes are the trust boundary of the agent stack, yet our empirical campaign shows the *control-plane trust inversion*—naming that promises restriction shipped as unauthenticated direct code execution—is a live, reproducible defect in freshly-published 0-star repositories, distinct from the MCP-tool hardening documented in prior work. Three runtime-confirmed disclosures (arbitrary command/Python execution with policy bypass; hardcoded credential plus unauthenticated services) contrast with a hardened control group, showing the ecosystem is two-sided. We provide detection signatures, honest deployment-boundary analysis, and hardening guidance, and argue that security review of agent control planes must treat the name as a claim, not a guarantee.

10 Disclosure & Ethics

All three findings were runtime-verified on isolated loopback instances with marker commands only (no real credentials probed, no harm performed), then disclosed via public GitHub issues with honest deployment-boundary scope. No private data, credentials, or production systems were accessed. The CVE-798 hardcoded-key finding’s validity was explicitly *not* probed against the external service.

References

1. Agent-to-agent (a2a) protocol. <https://a2a-protocol.org>, accessed 2026-10-02
2. Model context protocol specification. <https://modelcontextprotocol.io/specification>, accessed 2026-10-02
3. Agent execution control planes: Empirical study of freshly-published control-plane repositories. <https://zenodo.org/doi/10.5281/zenodo.PENDING> (2026), zenodo preprint (this work)
4. Credential-theft oracles, principal-spoofing, and unauthenticated write surfaces in freshly-published mcp and a2a servers. <https://zenodo.org/doi/10.5281/zenodo.22973907> (2026), zenodo preprint
5. Security analysis of the mcp ecosystem: From protocol design to production vulnerabilities. <https://zenodo.org/doi/10.5281/zenodo.22942676> (2026), zenodo preprint